

AVVISO

Si informa che **lunedì 6 luglio**, a causa della migrazione del firewall di Ateneo, la piattaforma non sarà accessibile **dalle ore 8:00 alle ore 13:00**.



Ingegneria del Software

[Torna al corso](#)

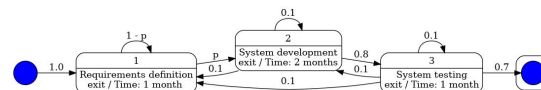
Stato	Completato
Iniziato	domenica, 5 luglio 2026, 11:37
Terminato	domenica, 5 luglio 2026, 11:54
Tempo impiegato	17 min. 31 secondi
Punteggio	47,00/50,00
Valutazione	94,00 su un massimo di 100,00

Domanda 1

Si consideri il processo software in figura

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 4 mesi sia almeno 0.5 ?

Scegli un'alternativa:

- ☒ 1. 0.893 ✓
- ☐ 2. 0.959
- ☐ 3. 0.645

Your answer is correct.

La risposta corretta è: 0.893

Domanda 2

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo di sviluppo plan-driven consiste di 2 fasi F1, F2, ciascuna costo A. Alla fine di ogni fase vengono prese in considerazione le "change requests" e, se ve ne sono, lo sviluppo viene ripetuto a partire dalla prima iterazione. Quindi con nessuna change request si hanno le fasi: F1, F2 e costo 2A. Con una "change request" dopo la prima fase si ha: F1, F1, F2 e costo 3A. Con una change request dopo la fase 2 si ha: F1, F2, F1, F2 e costo 4A. Qual'è il costo nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2.

Scegli un'alternativa:

- ☐ 1. 6*A
- ☒ 2. 5*A ✓
- ☐ 3. 7*A

Your answer is correct.

Nel caso in cui ci siano change requests sia dopo la fase 1 che dopo la fase 2 si ha: F1, F1, F2, F1, F2. Il costo quindi è 5*A.

La risposta corretta è: 5*A

Domanda 3

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x1 rappresenta lo stato precedente ad x e la variabile x2 lo stato precedente ad x1.

Si consideri il seguente requisito:

La differenza tra il valore dello stato presente e di quello precedente è compresa tra -1 ed 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (-1 <= (x - x1)) && ((x1 - x) <= 1) )`
- ☒ b. `assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )` ✓
- ☐ c. `assert( (x - x1) <= 1 )`

Your answer is correct.

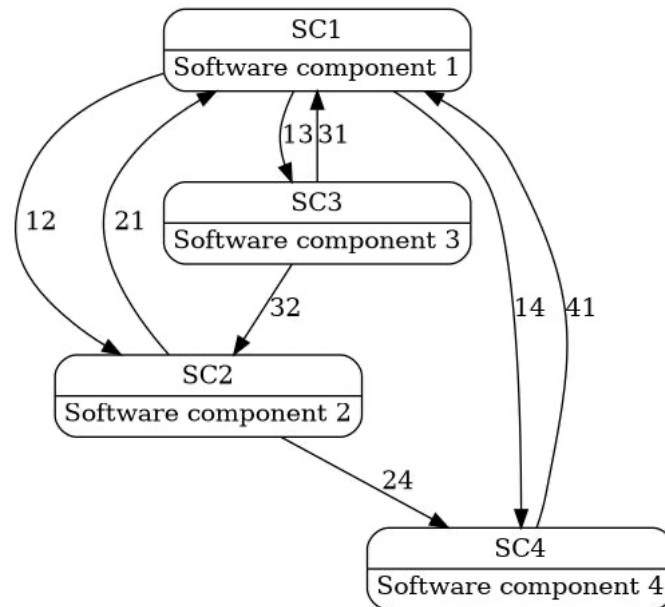
La risposta corretta è:
`assert( (-1 <= (x - x1)) && ((x - x1) <= 1) )`

Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente architettura software:



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☐ 1. // Begin input/output connections for system components
 Input 13 of SC3 = Output 13 of SC1
 Input 14 of SC4 = Output 14 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 24 of SC4 = Output 24 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 32 of SC2 = Output 32 of SC3
 Input 34 of SC4 = Output 34 of SC3
 Input 41 of SC1 = Output 41 of SC4
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components
- ☐ 2. // Begin input/output connections for system components
 Input 12 of SC2 = Output 12 of SC1
 Input 14 of SC4 = Output 14 of SC1
 Input 23 of SC3 = Output 23 of SC2
 Input 31 of SC1 = Output 31 of SC3
 Input 32 of SC2 = Output 32 of SC3
 Input 41 of SC1 = Output 41 of SC4
 // End input/output connections for system components

- ☒ 3. // Begin input/output connections for system components ✔ Correct.
- Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 41 of SC1 = Output 41 of SC4
// End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 14 of SC4 = Output 14 of SC1
Input 21 of SC1 = Output 21 of SC2
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 41 of SC1 = Output 41 of SC4
// End input/output connections for system components

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

"Ogni giorno, per ciascuna clinica, il sistema genererà una lista dei pazienti che hanno un appuntamento quel giorno."

La frase precedente è un esempio di:

Scegli un'alternativa:

- ☒ 1. Requisito funzionale. ✔
- ☐ 2. Requisito di performance.
- ☐ 3. Requisito non-funzionale.

Your answer is correct.

La risposta corretta è: Requisito funzionale.

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P * C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b * S)$, dove b è una opportuna costante note da dati storici aziendali.

- $b = 0.000020$
- $C = 812937.000000$
- $R = 1082.000000$

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☒ 1. Costo: 326458  Correct.
- ☐ 2. Costo: 464378
- ☐ 3. Costo: 380356

Your answer is correct.

La risposta corretta è: Costo: 326458

Domanda 7

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri la seguente funzione di transizione.

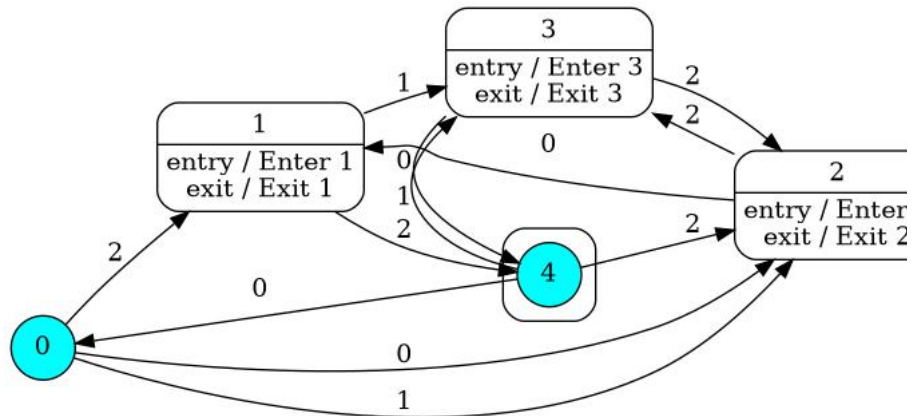
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (2);
        case 1: return (2);
        case 2: return (1);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (4);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (1);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (4);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (0);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    default: return (0);
  }
} // f()
```

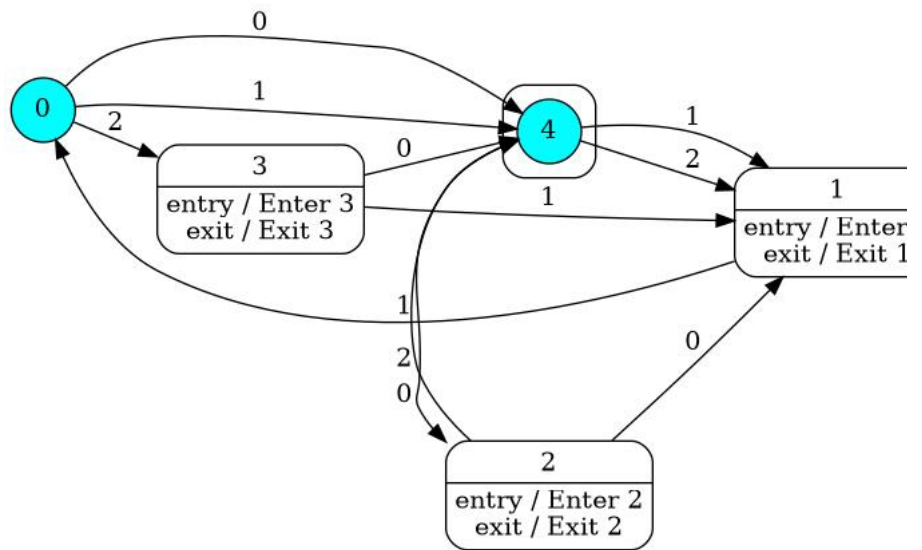
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

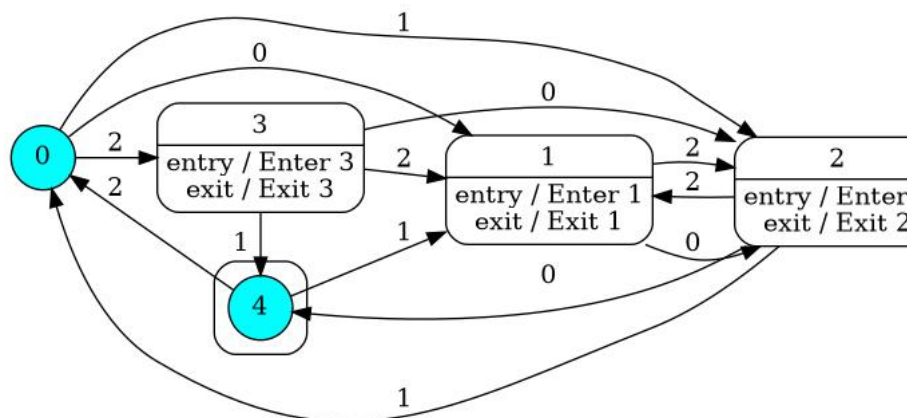
☒ 1.



☐ 2.

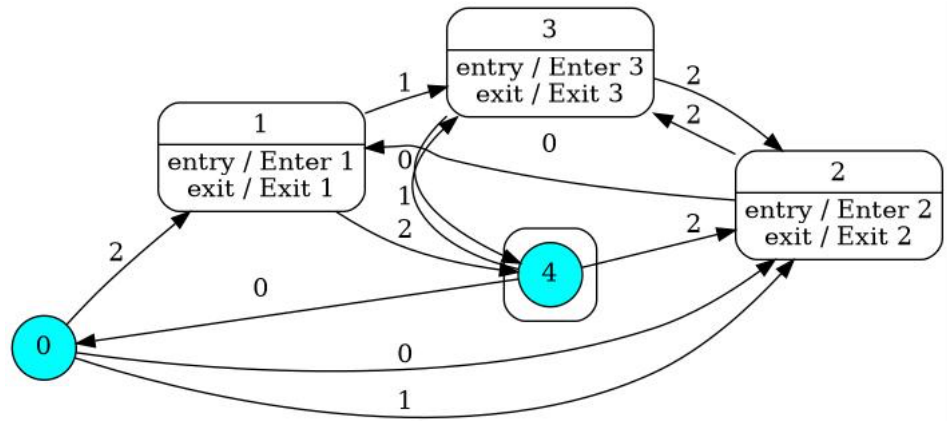


☐ 3.



Your answer is correct.

La risposta corretta è:



Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti affermazioni è vera riguardo ai metodi agile ?

Scegli un'alternativa:

- ☐ 1. I metodi agile sono metodi di sviluppo plan-driven.
- ☐ 2. I metodi agile sono metodi di sviluppo orientato al riuso.
- ☒ 3. I metodi agile sono metodi di sviluppo incrementale. ✓

Your answer is correct.

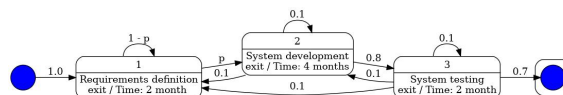
La risposta corretta è: I metodi agile sono metodi di sviluppo incrementale.

Domanda 9

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il processo software in figura



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini in esattamente 8 mesi sia almeno 0.5 ?

Scegli un'alternativa:

- ☒ 1. 0.893 ✓
- ☐ 2. 0.959
- ☐ 3. 0.645

Your answer is correct.

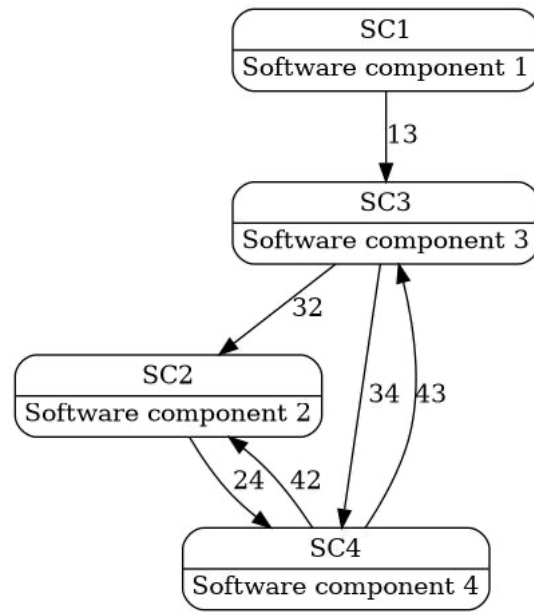
La risposta corretta è: 0.893

Domanda 10

Si consideri la seguente architettura software:

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Quale dei seguenti frammenti di pseudo codice meglio la rappresenta?

Scegli un'alternativa:

- ☐ 1. // Begin input/output connections for system components
 Input 13 of SC3 = Output 13 of SC1
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 41 of SC1 = Output 41 of SC4
 Input 42 of SC2 = Output 42 of SC4
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components
- ☒ 2. // Begin input/output connections for system components ✔ Correct.
 Input 13 of SC3 = Output 13 of SC1
 Input 24 of SC4 = Output 24 of SC2
 Input 32 of SC2 = Output 32 of SC3
 Input 34 of SC4 = Output 34 of SC3
 Input 42 of SC2 = Output 42 of SC4
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components
- ☐ 3. // Begin input/output connections for system components
 Input 21 of SC1 = Output 21 of SC2
 Input 23 of SC3 = Output 23 of SC2
 Input 24 of SC4 = Output 24 of SC2
 Input 32 of SC2 = Output 32 of SC3
 Input 34 of SC4 = Output 34 of SC3
 Input 41 of SC1 = Output 41 of SC4
 Input 43 of SC3 = Output 43 of SC4
 // End input/output connections for system components

Your answer is correct.

La risposta corretta è: // Begin input/output connections for system components

Input 13 of SC3 = Output 13 of SC1

Input 24 of SC4 = Output 24 of SC2

Input 32 of SC2 = Output 32 of SC3

Input 34 of SC4 = Output 34 of SC3

Input 42 of SC2 = Output 42 of SC4

Input 43 of SC3 = Output 43 of SC4

// End input/output connections for system components

Domanda 11

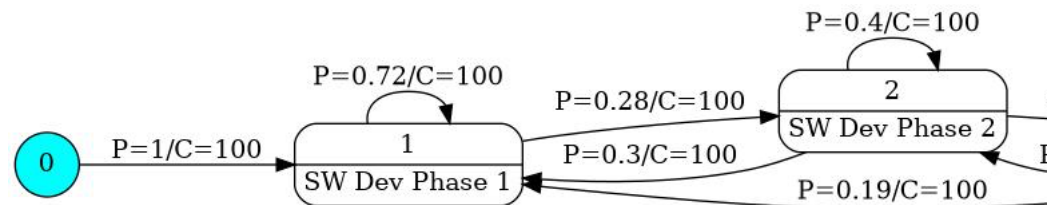
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 1"?

Scegli un'alternativa:

- ☐ 1. 14.0616
- ☐ 2. 19.4803
- ☒ 3. 3.57143 ✔ Correct.

Your answer is correct.

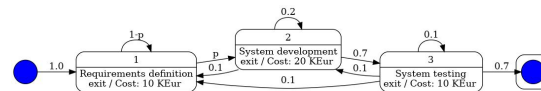
La risposta corretta è: 3.57143

Domanda 12

Si consideri il processo software in figura

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini con un costo di esattamente 40K EUR sia almeno 0.5.

Scegli un'alternativa:

- ☐ 1. 0.785
- ☒ 2. 0.98 ✓
- ☐ 3. 0.538

Your answer is correct.

La risposta corretta è: 0.98

Domanda 13

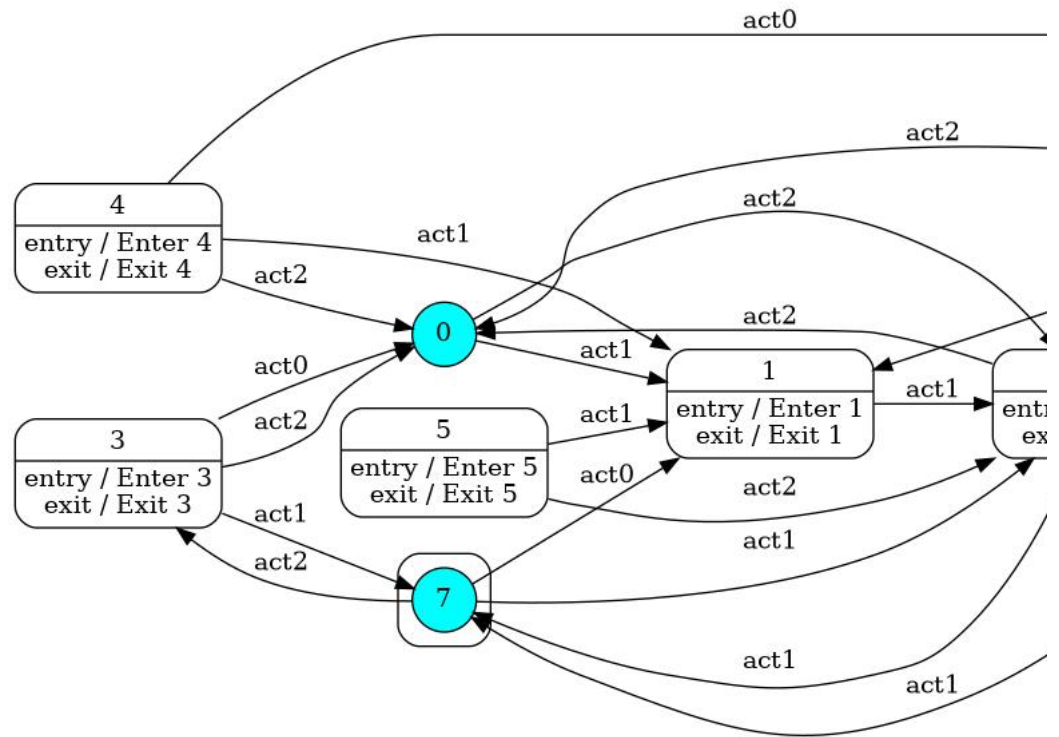
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act1 act1 act0 act2 act2

Test case 2: act2 act2 act1 act1

Test case 3: act2 act1

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. Transition coverage: 0.734
- ☐ 2. Transition coverage: 0.189
- ☒ 3. Transition coverage: 0.300 Correct.

Your answer is correct.

La risposta corretta è: Transition coverage: 0.300

Domanda 14

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se il sistema è nello stato 0 allora dopo al più due transizioni sarà nello stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x==0) ? (x2==2) : 1 )`
- ☐ b. `assert( (x==0) ? (x2==2) : 0 )`
- ☒ c. `assert( (x2==0) ? (x==2) : 1 )` ✓

Your answer is correct.

La risposta corretta è:
`assert( (x2==0) ? (x==2) : 1 )`

Domanda 15

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
5	0
5	1
5	2
5	3
5	4

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	6
1	4
2	1
3	1
4	4
5	4

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla(i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 1 | 3 |
| 0 | 3 | 1 |
| 1 | 4 | 4 |
| 2 | 1 | 1 |
| 2 | 2 | 1 |
| 3 | 5 | 4 |
| 5 | 0 | 4 |
| 6 | 0 | 2 |
- ☒ 2.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 2 | 1 |
| 0 | 3 | 1 |
| 0 | 4 | 2 |
| 1 | 0 | 2 |
| 1 | 4 | 2 |
| 2 | 1 | 4 |
| 3 | 0 | 4 |
| 4 | 5 | 4 |
- Correct.

○ 3.

Time slot (Month)	Task	Effort (PM)
0	1	1
0	4	3
1	0	2
1	3	1
1	4	1
2	1	3
2	2	1
3	0	4
4	5	3

Your answer is correct.

La risposta corretta è:

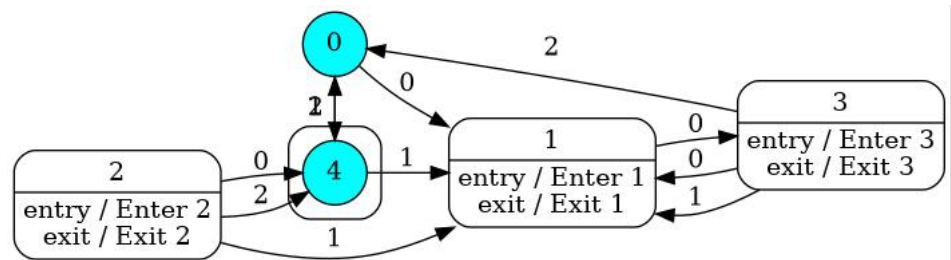
Time slot (Month)	Task	Effort (PM)
0	2	1
0	3	1
0	4	2
1	0	2
1	4	2
2	1	4
3	0	4
4	5	4

Domanda 16

Si consideri lo state diagram in figura.

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

- ☐ 1. `#include "main.h"`
- ```
int f(int x, int u)
{ switch (x) {
 case 0: switch (u) {
 case 0: return (2);
 case 1: return (0);
 case 2: return (0);
 default: return (0); }
 case 1: switch (u) {
 case 0: return (4);
 case 1: return (2);
 case 2: return (2);
 default: return (0); }
 case 2: switch (u) {
 case 0: return (1);
 case 1: return (2);
 case 2: return (4);
 default: return (0); }
 case 3: switch (u) {
 case 0: return (2);
 case 1: return (2);
 case 2: return (3);
 default: return (0); }
 case 4: switch (u) {
 case 0: return (0);
 case 1: return (4);
 case 2: return (4);
 default: return (0); }
 default: return (0);
}
} // f()
```
- ☐ 2. `#include "main.h"`
- ```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (3);
    case 1: return (0);
    case 2: return (3);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (3);
    case 1: return (3);
    case 2: return (0);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (0);
    case 1: return (3);
    case 2: return (2);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (4);
    case 1: return (4);
    case 2: return (3);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (0);
    case 1: return (4);
    case 2: return (4);
    default: return (0); }
  default: return (0);
}
} // f()
```

3. `#include "main.h"` ✔ Correct.

```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    default: return (0);
} } // f()
```

Your answer is correct.

La risposta corretta è:

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (1);
        case 1: return (4);
        case 2: return (0);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (1);
        case 2: return (1);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (0);
        default: return (0); }
    default: return (0);
} } // f()
```

Domanda 17

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive il criterio di "requirements verifiability" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Per ciascuna componente del sistema, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che essa soddisfa tutti i requisiti.
- ☐ 2. Per ciascuna coppia di componenti, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che l'interazione tra le componenti soddisfa tutti i requisiti di interfaccia.
- ☒ 3. Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato. 

Your answer is correct.

La risposta corretta è: Per ciascun requisito, dovremmo essere in grado di scrivere un insieme di test che può dimostrare che il sistema sviluppato soddisfa il requisito considerato.

Domanda 18

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La WBS (Work Breakdown Structure) di un progetto consiste di 6 tasks: T0, T1, T2, T3, T4, T5.

I vincoli di precedenza tra tasks sono dati nella *tabella delle precedenze* definita nel seguito.

ID Successor Task	ID Predecessor Task
3	0
3	1
3	2
4	0
4	1
4	2
5	3
5	4

La riga (i, j) nella tabella delle precedenze indica che il task j precede il task i (cioè che il task i può iniziare solo dopo che il task j è terminato).

L'effort (in PM, Person-Months) da dedicare a ciascun task per completare il progetto è dato dalla *tabella degli efforts* definita nel seguito:

Task ID	Effort
0	2
1	6
2	4
3	1
4	2

5	3
---	---

La riga (i, k) nella tabella degli efforts indica che il completamento del task i richiede un effort di k PM.

Lo scheduling per i tasks è definito attraverso una *tabella di scheduling*. Ciascuna riga della tabella di scheduling consiste di una tripla (i, j, k) dove:

1. i è il time slot (mese, nel caso in cui l'effort è misurato in PM);
2. j è il task
3. k è l'effort (positivo) dedicato al task j nel time slot i

Nella tabella di scheduling sono presenti solo le triple in cui l'effort è positivo. Per tutte le altre triple si intende che l'effort è 0.

Un esempio di *tabella di scheduling* è il seguente:

Time slot	Task	Effort (PM)
0	0	2
0	2	1
1	0	5
2	2	3

Nella tabella sopra al task T0 sono dedicati in totale 7 PM (2 nel time slot 0 e 5 nel time slot 1) ed al task T2 sono dedicati in totale 4 PM (1 nel time slot 0 e 3 nel time slot 2).

Nel time slot 0 l'effort totale è 3 (2PM su T0 ed 1 PM su T2)

Nel time slot 1 l'effort totale è 5 (5 PM su T0).

Nel time slot 2 l'effort totale è 3 (3 PM su T2).

Disponedo di 4 unità di personale (e quindi per ogni time slot l'effort totale può essere al più 4), quali delle seguenti tabelle di scheduling soddisfa i vincoli espressi dalle tabelle delle precedenze e degli efforts definite sopra?

Scegli un'alternativa:

- ☐ 1.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 2 | 4 |
| 1 | 0 | 2 |
| 1 | 1 | 2 |
| 2 | 1 | 4 |
| 3 | 3 | 1 |
| 4 | 4 | 1 |
| 5 | 5 | 3 |

- ☐ 2.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 1 | 2 |
| 0 | 2 | 2 |
| 1 | 1 | 4 |
| 2 | 2 | 2 |
| 3 | 4 | 2 |
| 5 | 0 | 2 |
| 7 | 3 | 1 |
| 9 | 5 | 3 |
- ☒ 3.
- | Time slot (Month) | Task | Effort (PM) |
|-------------------|------|-------------|
| 0 | 2 | 4 |
| 1 | 1 | 4 |
| 2 | 0 | 2 |
| 2 | 1 | 2 |
| 3 | 4 | 2 |
| 4 | 3 | 1 |
| 5 | 5 | 3 |
- Correct.

Your answer is correct.

La risposta corretta è:

Time slot (Month)	Task	Effort (PM)
0	2	4
1	1	4
2	0	2
2	1	2
3	4	2
4	3	1
5	5	3

Domanda 19

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Focalizzandosi sui metodi agile di sviluppo del software, quale delle seguenti affermazioni è vera?

Scegli un'alternativa:

- ☒ 1. Le attività di definizione dei requisiti e di sviluppo sono interleaved. ✓
- ☐ 2. Per evitare di sprecare tempo durante la fase di sviluppo del software, questa inizia solo quando i requisiti sono stati completamente definiti.
- ☐ 3. Per evitare di sprecare tempo durante la fase di sviluppo del software, il customer non è mai coinvolto nel processo di sviluppo del software.

Your answer is correct.

La risposta corretta è: Le attività di definizione dei requisiti e di sviluppo sono interleaved.

Domanda 20

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "validity check" che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che i requisiti funzionali descrivano tutte le funzionalità del sistema.
- ☒ 2. Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer". 
- ☐ 3. Assicurarsi che non ci siano requisiti in conflitto con altri requisiti.

Your answer is correct.

La risposta corretta è: Assicurarsi che un sistema che soddisfa i requisiti risolve il problema del "customer".

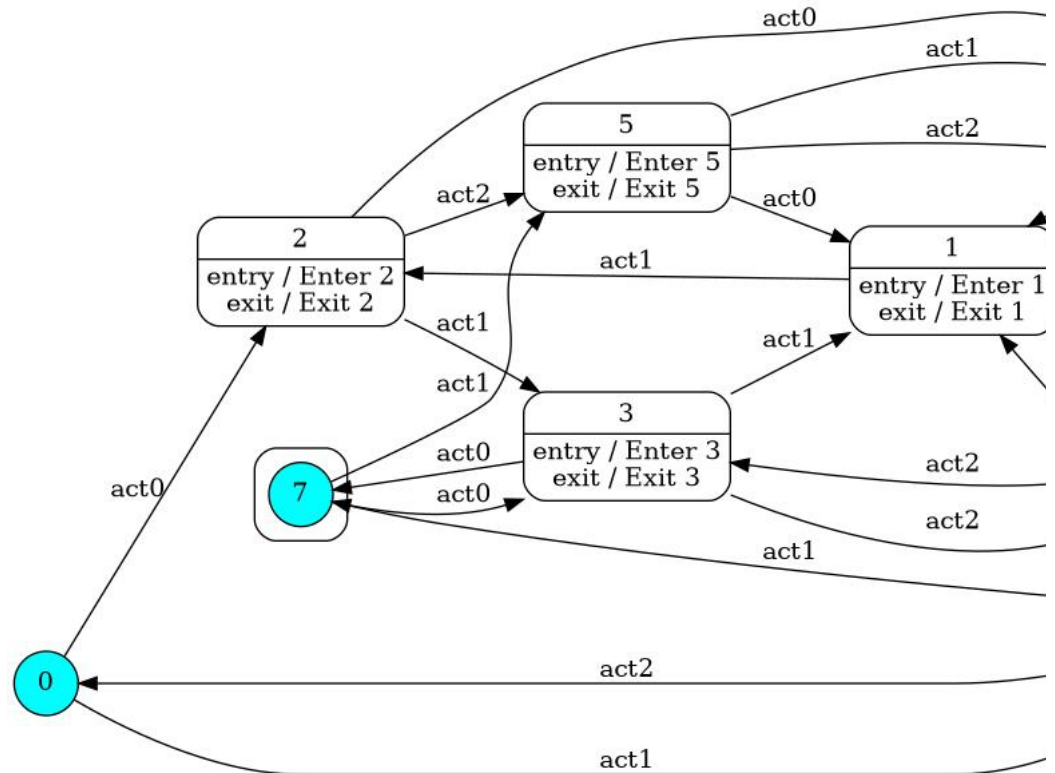
Domanda 21

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act0 act2 act1 act0 act1

Test case 2: act0 act0 act0 act1 act1

Test case 3: act0 act0 act0 act2 act2

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.923
- ☒ 2. State coverage: 0.750 ✓ Correct.
- ☐ 3. State coverage: 0.852

Your answer is correct.

La risposta corretta è: State coverage: 0.750

Domanda 22

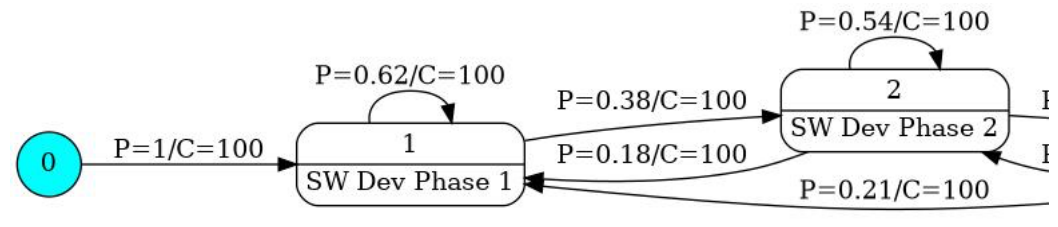
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 2"?

Scegli un'alternativa:

- ☐ 1. 0.442468
- ☐ 2. 0.752675
- ☒ 3. 1.17391 Correct.

Your answer is correct.

La risposta corretta è: 1.17391

Domanda 23

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.

La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.

Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.

```
int f(int x[3])
{
    if ((-1)*x[0] + (0)*x[1] + (4)*x[2] <= -4)
        if ((-1)*x[0] + (-4)*x[1] + (-4)*x[2] <= -4)
            {return (0); }
        else
            {return (1); }
    else if ((0)*x[0] + (-1)*x[1] + (-3)*x[2] <= -4)
        if ((3)*x[0] + (0)*x[1] + (1)*x[2] <= 3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = -1, x[1] = 1, x[2] = 5.

Test 2: x[0] = 5, x[1] = 1, x[2] = 2.

Test 3: x[0] = 3, x[1] = 3, x[2] = 0.

Test 4: x[0] = 2, x[1] = 3, x[2] = -4.

Scegli un'alternativa:

- ☐ 1. Coverage: 0.521
- ☐ 2. Coverage: 0.185
- ☒ 3. Coverage: 0.800  Correct.

Your answer is correct.

La risposta corretta è: Coverage: 0.800

Domanda 24

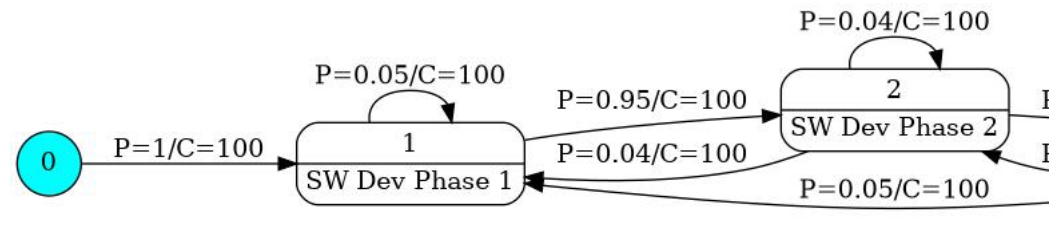
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.0695 ✔ Correct.
- ☐ 2. 0.364118
- ☐ 3. 0.690644

Your answer is correct.

La risposta corretta è: 0.0695

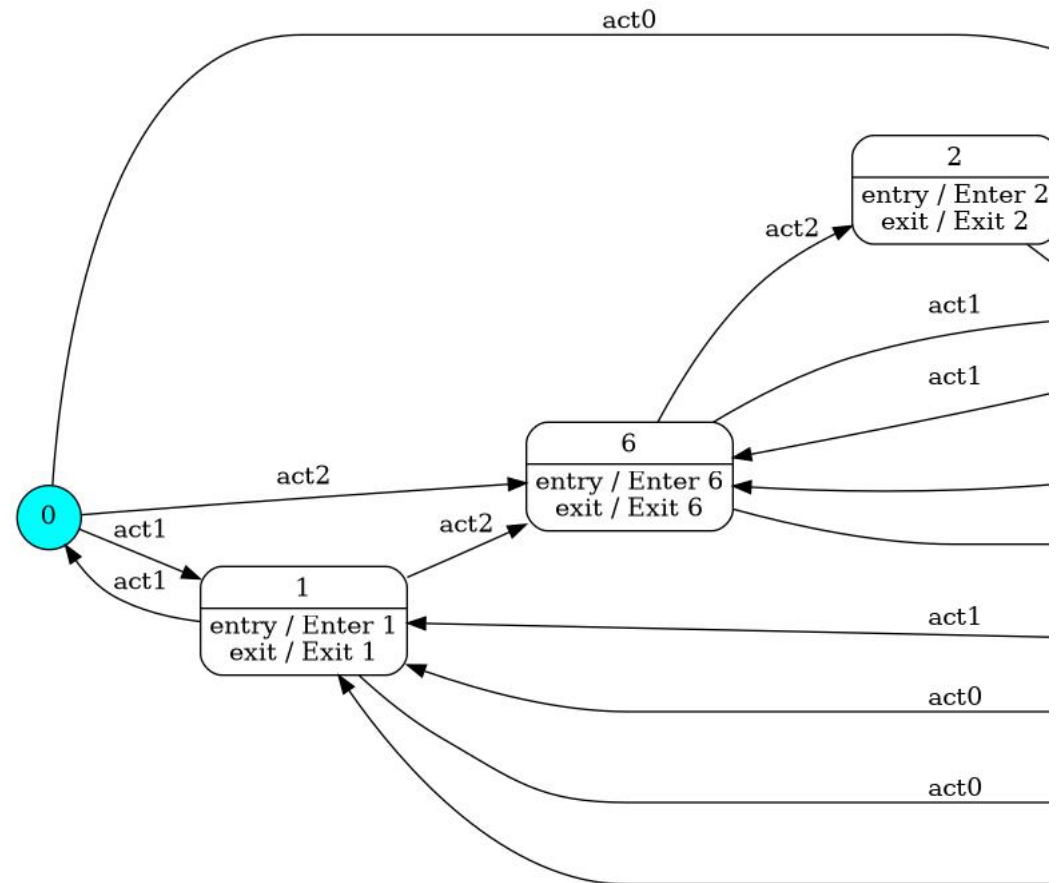
Domanda 25

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La state coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di stati (inclusi START ed END) raggiunti almeno una volta.

Si consideri lo state diagram UML in figura:



Si consideri il seguente insieme di test cases:

Test case 1: act2 act0 act2 act1 act2

Test case 2: act2 act0 act0 act0 act2

Test case 3: act2 act0 act1

Quale delle seguenti è la migliore stima della state coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☐ 1. State coverage: 0.476
- ☐ 2. State coverage: 0.860
- ☒ 3. State coverage: 0.625 Correct.

Your answer is correct.

La risposta corretta è: State coverage: 0.625

Domanda 26

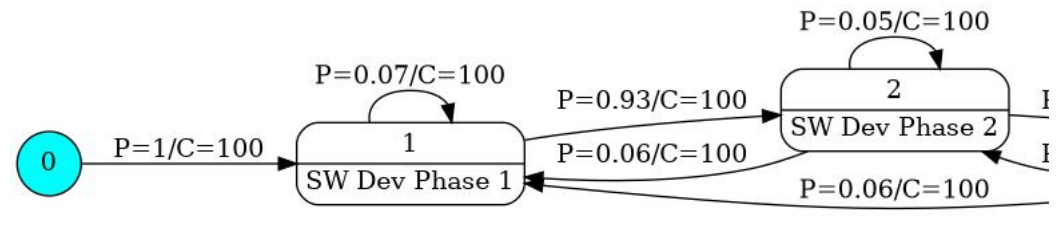
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.564731
- ☐ 2. 0.971791
- ☒ 3. 0.720099 ✔ Correct.

Your answer is correct.

La risposta corretta è: 0.720099

Domanda 27

Risposta corretta

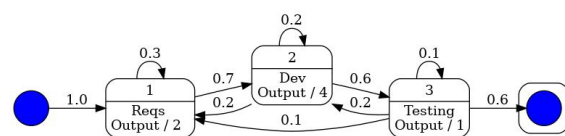
Punteggio ottenuto 1,00 su 1,00

Un processo software di n stati può essere modellato con una coppia (A, Y) dove:

- A è una matrice che definisce una catena di Markov con n stati etichettati con interi da 0 ad $n-1$. Lo stato iniziale è etichettato con 0 mentre lo stato finale con $n-1$.

- Y è un vettore di n elementi che definisce l'output della catena di Markov associando un output (se definito) a ciascuno stato.

Quale delle seguenti rappresentazioni (A, Y) meglio modella il processo software in figura ?



Scegli un'alternativa:

☐ 1. $A[0][1] = 1.0$

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 2$

$Y[3] = 1$

$Y[4] = -$

☐ 2. $A[0][1] = 1.0$

$A[1][1] = 0.4$

$A[1][2] = 0.6$

$A[2][1] = 0.1$

$A[2][2] = 0.1$

$A[2][3] = 0.8$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 4$

$Y[3] = 1$

$Y[4] = -$

☒ 3. $A[0][1] = 1.0$ 

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 4$

$Y[3] = 1$

$Y[4] = -$

Your answer is correct.

La risposta corretta è:

$A[0][1] = 1.0$

$A[1][1] = 0.3$

$A[1][2] = 0.7$

$A[2][1] = 0.2$

$A[2][2] = 0.2$

$A[2][3] = 0.6$

$A[3][1] = 0.1$

$A[3][2] = 0.2$

$A[3][3] = 0.1$

$A[3][4] = 0.6$

$A[4][4] = 1.0$

$Y[0] = -$

$Y[1] = 2$

$Y[2] = 4$

$Y[3] = 1$

$Y[4] = -$

Domanda 28

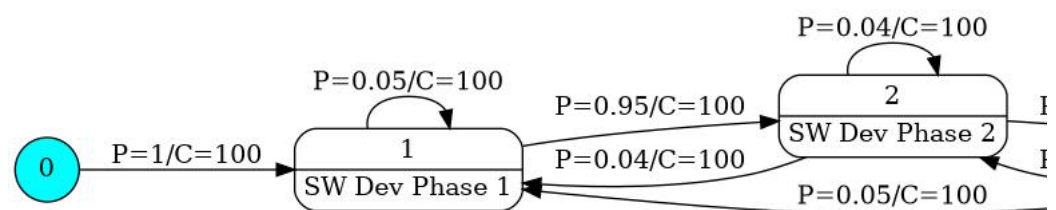
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 400.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.75164 ✔ Correct.
- ☐ 2. 0.213091
- ☐ 3. 0.415743

Your answer is correct.

La risposta corretta è: 0.75164

Domanda 29

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale dei seguenti è un requisito non funzionale?

Scegli un'alternativa:

- ☐ 1. L'output del sistema è sempre una matrice invertibile.
- ☒ 2. L'utilizzo della RAM è sempre inferiore ai 4GBytes.  Correct.
- ☐ 3. L'output del sistema è sempre un vettore ordinato.

Your answer is correct.

La risposta corretta è: L'utilizzo della RAM è sempre inferiore ai 4GBytes.

Domanda 30

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un *Path* in un programma è una sequenza di istruzioni dall'inizio alla fine del programma.**La *Path Coverage* di un insieme di test cases è la percentuale di path del programma che sono attraversati da almeno un test case.****Considerando la seguente funzione C, qual'è la path coverage con i tests cases dati.**

```
int f(int x[3])
{
    if ((-2)*x[0] + (3)*x[1] + (-1)*x[2] <= 1)
        if ((-3)*x[0] + (3)*x[1] + (-2)*x[2] <= -1)
            {return (0); }
        else
            {return (1); }
    else if ((0)*x[0] + (-2)*x[1] + (0)*x[2] <= 4)
        if ((0)*x[0] + (-2)*x[1] + (-3)*x[2] <= 3)
            {return (2); }
        else
            {return (3); }
    else
        {return (4); }
} /* f() */
```

Test 1: x[0] = 3, x[1] = 3, x[2] = 0.**Test 2: x[0] = 0, x[1] = -1, x[2] = 2.****Test 3: x[0] = 2, x[1] = 1, x[2] = -3.**

Scegli un'alternativa:

- ☒ 1. Coverage: 0.600  Correct.
- ☐ 2. Coverage: 0.731
- ☐ 3. Coverage: 0.960

Your answer is correct.

La risposta corretta è: Coverage: 0.600

Domanda 31

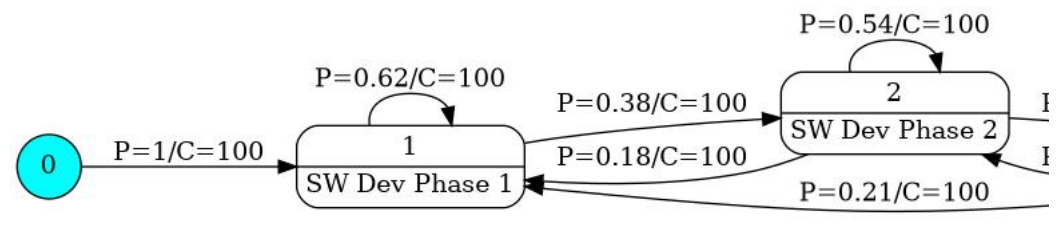
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 3"?

Scegli un'alternativa:

- ☒ 1. 0.538462 ✔ Correct.
- ☐ 2. 0.164015
- ☐ 3. 1.50332

Your answer is correct.

La risposta corretta è: 0.538462

Domanda 32

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile **x** rappresenta lo stato presente del sistema, la variabile **x1** rappresenta lo stato precedente ad **x** e la variabile **x2** lo stato precedente ad **x1**.

Si consideri il seguente requisito:

Se il sistema è nello stato 1 allora o permane nello stato di 1 oppure passa allo stato 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert((x==1) ? ((x1==1) || (x1==2)) : 1)`
- ☒ b. `assert((x1==1) ? ((x==1) || (x==2)) : 1)` ✔
- ☐ c. `assert((x1 == 1) ? ((x==1) || (x==2)) : 0)`

Your answer is correct.

La risposta corretta è:

`assert((x1==1) ? ((x==1) || (x==2)) : 1)`

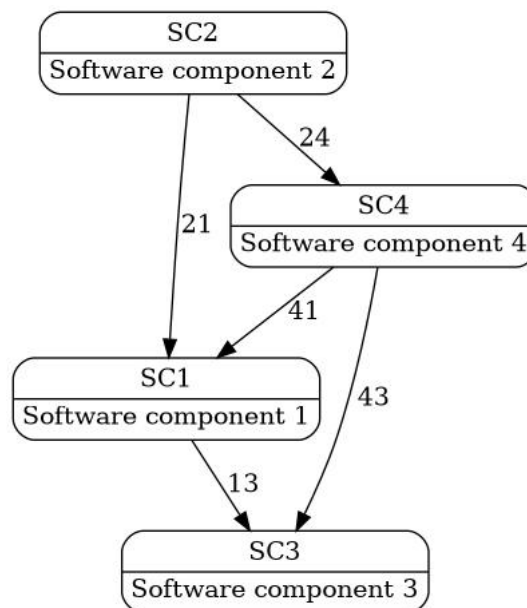
Domanda 33

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

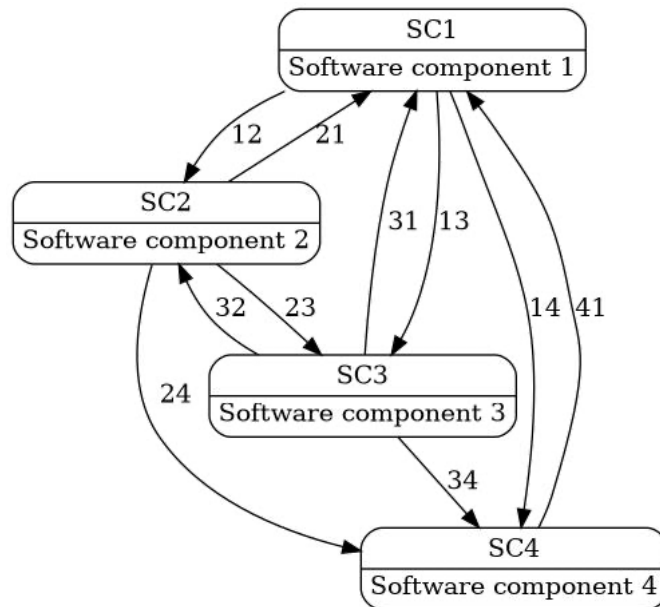
Si consideri il seguente frammento di pseudo codice.**// Begin input/output connections for system components****Input 12 of SC2 = Output 12 of SC1****Input 13 of SC3 = Output 13 of SC1****Input 14 of SC4 = Output 14 of SC1****Input 21 of SC1 = Output 21 of SC2****Input 23 of SC3 = Output 23 of SC2****Input 24 of SC4 = Output 24 of SC2****Input 31 of SC1 = Output 31 of SC3****Input 32 of SC2 = Output 32 of SC3****Input 34 of SC4 = Output 34 of SC3****Input 41 of SC1 = Output 41 of SC4****// End input/output connections for system components****Quale delle seguenti architetture software meglio lo rappresenta ?**

Scegli un'alternativa:

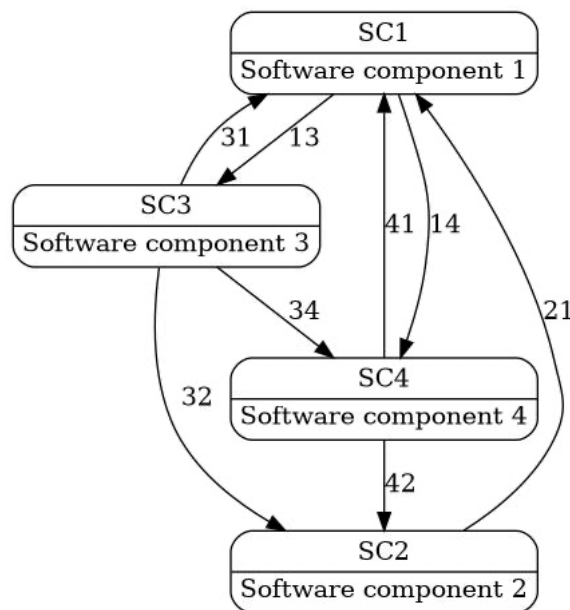
☐ 1.

2.

Correct.

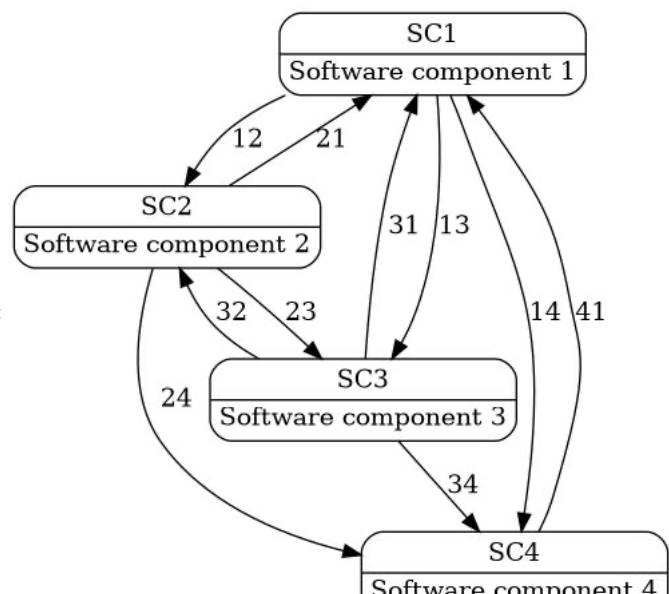


3.



Your answer is correct.

La risposta corretta è:

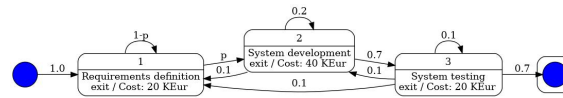


Domanda 34

Si consideri il processo software in figura

Risposta corretta

Punteggio ottenuto 1,00 su 1,00



Qual'è la migliore stima per il valore minimo di p che garantisce che la probabilità che il processo termini con un costo di esattamente 80K EUR sia almeno 0.5.

Scegli un'alternativa:

- ☐ 1. 0.785
- ☒ 2. 0.98 ✓
- ☐ 3. 0.538

Your answer is correct.

La risposta corretta è: 0.98

Domanda 35

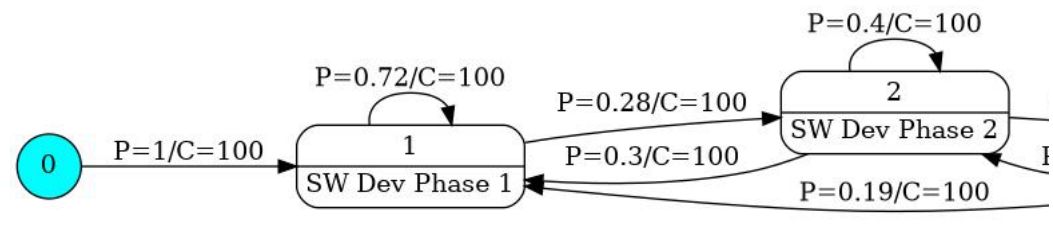
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 3"?

Scegli un'alternativa:

- ☐ 1. 6.85052
- ☒ 2. 1.81818 ✔ Correct.
- ☐ 3. 0.674053

Your answer is correct.

La risposta corretta è: 1.81818

Domanda 36

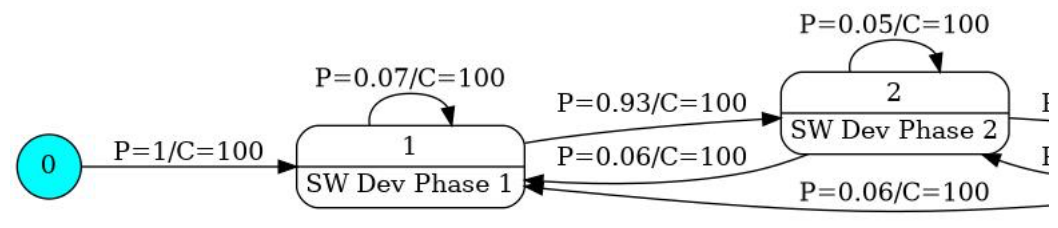
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 600.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.773429
- ☒ 2. 0.0701 ✓ Correct.
- ☐ 3. 0.0536754

Your answer is correct.

La risposta corretta è: 0.0701

Domanda 37

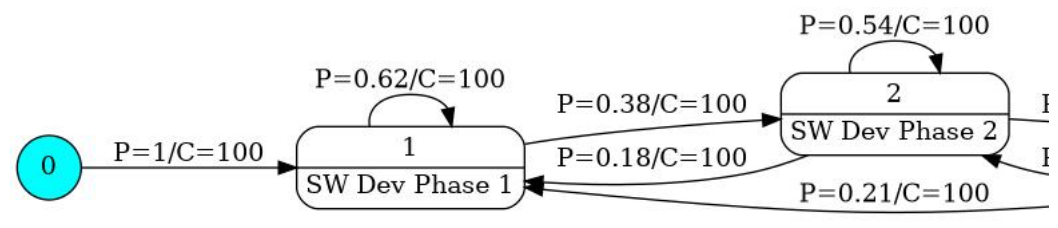
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di iterazioni della "SW Dev Phase 1"?

Scegli un'alternativa:

- ☐ 1. 4.95645
- ☒ 2. 1.63158 ✔ Correct.
- ☐ 3. 10.6293

Your answer is correct.

La risposta corretta è: 1.63158

Domanda 38

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si consideri il seguente frammento di pseudo codice.

```

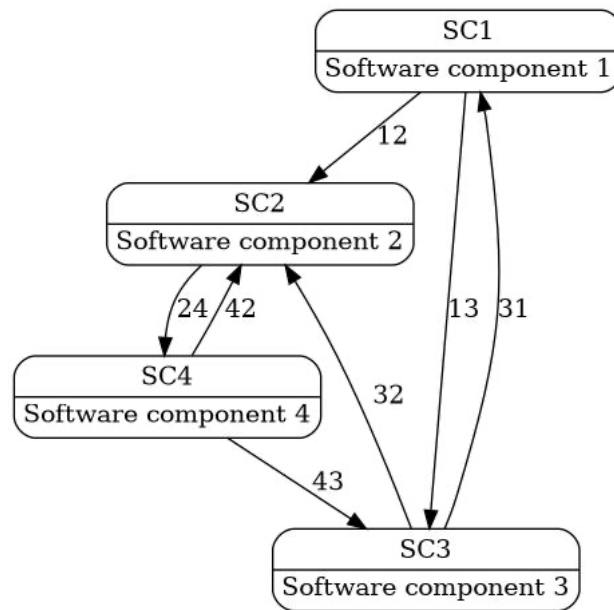
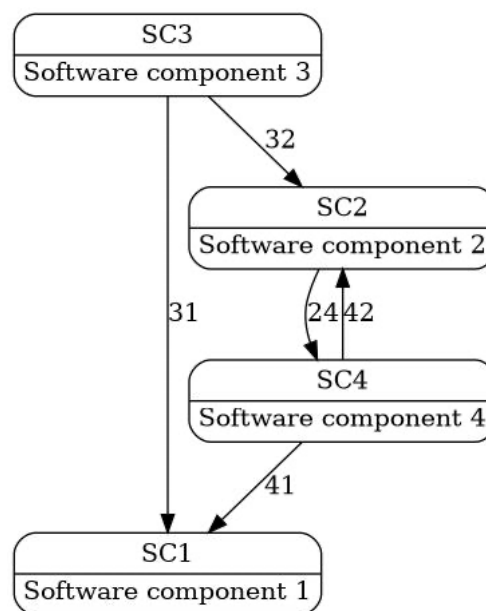
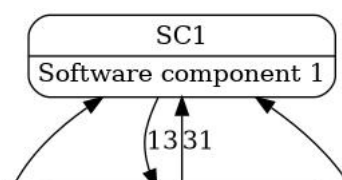
// Begin input/output connections for system components
Input 12 of SC2 = Output 12 of SC1
Input 13 of SC3 = Output 13 of SC1
Input 24 of SC4 = Output 24 of SC2
Input 31 of SC1 = Output 31 of SC3
Input 32 of SC2 = Output 32 of SC3
Input 42 of SC2 = Output 42 of SC4
Input 43 of SC3 = Output 43 of SC4
// End input/output connections for system components
  
```

Quale delle seguenti architetture software meglio lo rappresenta ?

Scegli un'alternativa:

☒ 1.

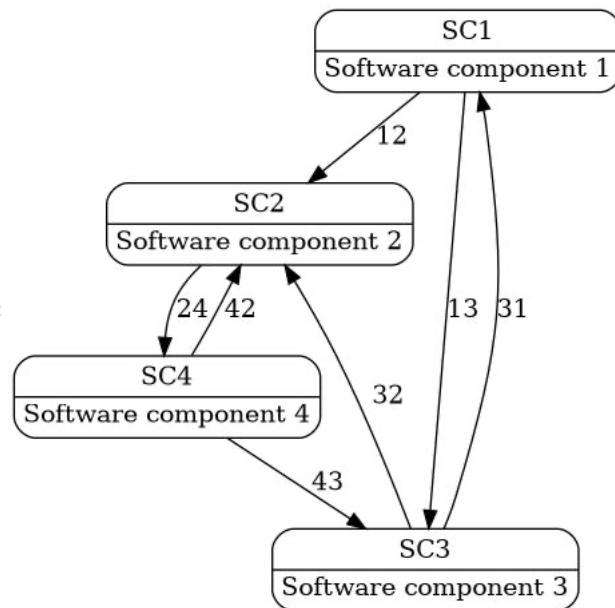
✔ Correct.

☐ 2.☐ 3.

/ (SC3) \

Your answer is correct.

La risposta corretta è:

**Domanda 39**

Risposta errata

Punteggio ottenuto -0,50 su 1,00

L'attività di *unit testing* di una funzione è stata suddivisa nei seguenti task:

Task identifier	Task description
TDEF	Definition of test cases
TORC	Definition of test oracle
TEXE	Esecuzione dei casi di test
TVAL	Valutazione dei casi di test

Quale dei seguenti insiemi di vincoli di precedenza (dove $A < B$ significa che A è preceduto da B) deve necessariamente valere, considerata la descrizione data dei tasks?

Scegli un'alternativa:

- ☒ 1. $TEXE < TORC$, $TVAL < TEXE$, $TVAL < TORC$. ⊗ Incorrect.
- ☐ 2. $TVAL < TEXE$, $TEXE < TDEF$, $TVAL < TORC$.
- ☐ 3. $TVAL < TEXE$, $TORC < TDEF$, $TVAL < TORC$.

Your answer is incorrect.

La risposta corretta è: $TVAL < TEXE$, $TEXE < TDEF$, $TVAL < TORC$.

Domanda 40

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Se lo stato vale 0 allora dopo al più 2 transizioni lo stato prende il valore 1.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☐ a. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 0 )`
- ☐ b. `assert( (x == 0) ? (x1 == 1) || (x2 == 1) : 1 )`
- ☒ c. `assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )` ✓

Your answer is correct.

La risposta corretta è:

`assert( (x2 == 0) ? (x1 == 1) || (x == 1) : 1 )`

Domanda 41

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Il rischio R può essere calcolato come $R = P \cdot C$, dove P è la probabilità dell'evento avverso (software failure nel nostro contesto) e C è il costo dell'occorrenza dell'evento avverso.

Assumiamo che la probabilità P sia legata al costo di sviluppo S dalla formula $P = \exp(-b \cdot S)$, dove b è una opportuna costante note da dati storici aziendali.

- **$b = 0.000106$**
- **$C = 488564.000000$**
- **$R = 349.000000$**

Quale delle seguenti opzioni meglio approssima il costo S per lo sviluppo del software in questione.

Scegli un'alternativa:

- ☐ 1. Costo: 70714
- ☐ 2. Costo: 82399
- ☒ 3. Costo: 68187 ✓ Correct.

Your answer is correct.

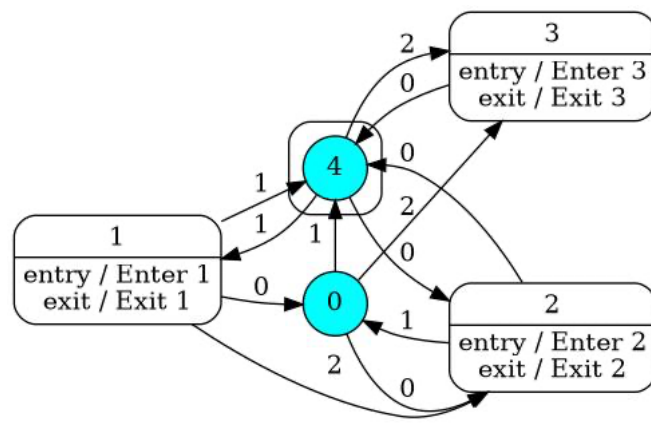
La risposta corretta è: Costo: 68187

Domanda 42

Risposta corretta

Si consideri lo state diagram in figura.

Punteggio ottenuto 1,00 su 1,00



Quale delle seguenti funzioni di transizione meglio lo rappresenta?

Scegli un'alternativa:

☐ 1. `#include "main.h"`

```
int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (4);
    case 1: return (0);
    case 2: return (1);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (3);
    case 1: return (1);
    case 2: return (1);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (0);
    case 1: return (1);
    case 2: return (2);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (3);
    case 1: return (0);
    case 2: return (0);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (1);
    case 1: return (3);
    case 2: return (1);
    default: return (0); }
  default: return (0);
} } // f()
```

- ☒ 2. `#include "main.h"` ✔ Correct.
- ```
int f(int x, int u)
{ switch (x) {
 case 0: switch (u) {
 case 0: return (2);
 case 1: return (4);
 case 2: return (3);
 default: return (0); }
 case 1: switch (u) {
 case 0: return (0);
 case 1: return (4);
 case 2: return (2);
 default: return (0); }
 case 2: switch (u) {
 case 0: return (4);
 case 1: return (0);
 case 2: return (2);
 default: return (0); }
 case 3: switch (u) {
 case 0: return (4);
 case 1: return (3);
 case 2: return (3);
 default: return (0); }
 case 4: switch (u) {
 case 0: return (2);
 case 1: return (1);
 case 2: return (3);
 default: return (0); }
 default: return (0);
}
} // f()
```
- ☐ 3. `#include "main.h"`
- ```
int f(int x, int u)
{ switch (x) {
    case 0: switch (u) {
        case 0: return (4);
        case 1: return (2);
        case 2: return (3);
        default: return (0); }
    case 1: switch (u) {
        case 0: return (3);
        case 1: return (3);
        case 2: return (3);
        default: return (0); }
    case 2: switch (u) {
        case 0: return (4);
        case 1: return (1);
        case 2: return (4);
        default: return (0); }
    case 3: switch (u) {
        case 0: return (1);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    case 4: switch (u) {
        case 0: return (2);
        case 1: return (3);
        case 2: return (2);
        default: return (0); }
    default: return (0);
}
} // f()
```

Your answer is correct.

La risposta corretta è:

```
#include "main.h"

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (2);
    case 1: return (4);
    case 2: return (3);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (0);
    case 1: return (4);
    case 2: return (2);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (4);
    case 1: return (0);
    case 2: return (2);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (4);
    case 1: return (3);
    case 2: return (3);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (2);
    case 1: return (1);
    case 2: return (3);
    default: return (0); }
  default: return (0);
}
} // f()
```

Domanda 43

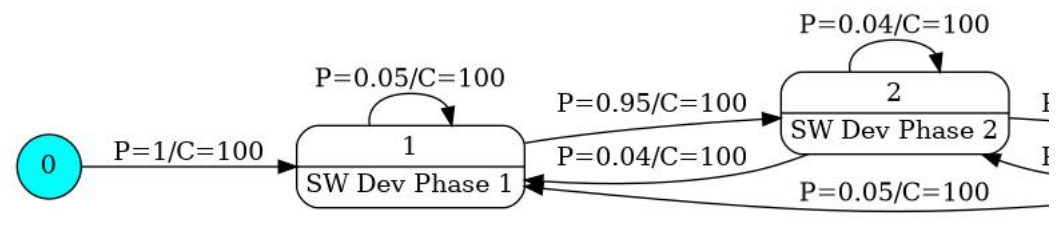
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☐ 1. 0.0487319
- ☒ 2. 0.0948 ✔ Correct.
- ☐ 3. 0.026554

Your answer is correct.

La risposta corretta è: 0.0948

Domanda 44

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software consiste delle seguenti fasi:

- 1: Requirements definition,**
- 2: System development,**
- 3: System testing.**
- 4: Delivey (fase finale)**

La probabilità di passare da una fase i ($i = 1, 2, 3$) alla fase successiva ($i+1$) è sempre 0.7, mentre con probabilità 0.3 è necessario ripetere la fase in corso a causa di errori.

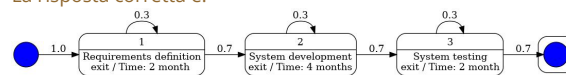
Quale delle seguenti catene di Markov meglio modella il processo software di cui sopra ?

Scegli un'alternativa:

- ☐ 1.
-
- ☒ 2.
-
- ☐ 3.
-

Your answer is correct.

La risposta corretta è:



Domanda 45

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Quale delle seguenti frasi meglio descrive l'obiettivo del "check di realismo" (realizability) che è parte della "requirements validation activity".

Scegli un'alternativa:

- ☐ 1. Assicurarsi che le funzionalità richieste al sistema siano necessarie per soddisfare le necessità del customer.
- ☒ 2. Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfa i requisiti. 
- ☐ 3. Assicurarsi che le performance richieste al sistema siano necessarie per soddisfare le necessità del customer.

Your answer is correct.

La risposta corretta è: Assicurarsi che, tenendo conto della tecnologia, budget e tempo disponibili, sia possibile realizzare un sistema che soddisfa i requisiti.

Domanda 46

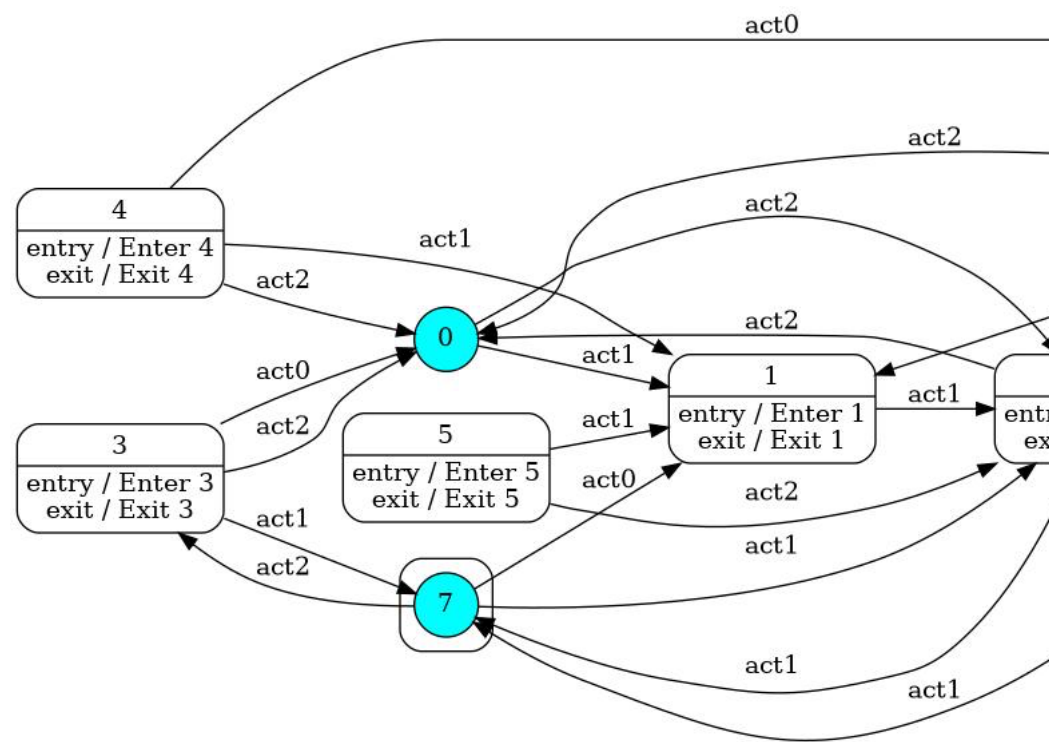
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La transition coverage di un insieme di test cases (cioè sequenze di inputs) per uno state diagram è la percentuale di transizioni (archi nel grafo dello state diagram) percorsi almeno una volta.

Si consideri lo state diagram in figura:

Si consideri lo state diagram UML in figura:



ed il seguente insieme di test cases:

Test case 1: act1 act1 act0 act2 act2

Test case 2: act2 act2 act1 act1

Test case 3: act2 act1

Quale delle seguenti è la migliore stima della transition coverage per i test cases di cui sopra?

Scegli un'alternativa:

- ☒ 1. Transition coverage: 0.300 Correct.
- ☐ 2. Transition coverage: 0.734
- ☐ 3. Transition coverage: 0.189

Your answer is correct.

La risposta corretta è: Transition coverage: 0.300

Domanda 47

Risposta errata

Si consideri la seguente funzione di transizione.

Punteggio ottenuto -0,50 su 1,00

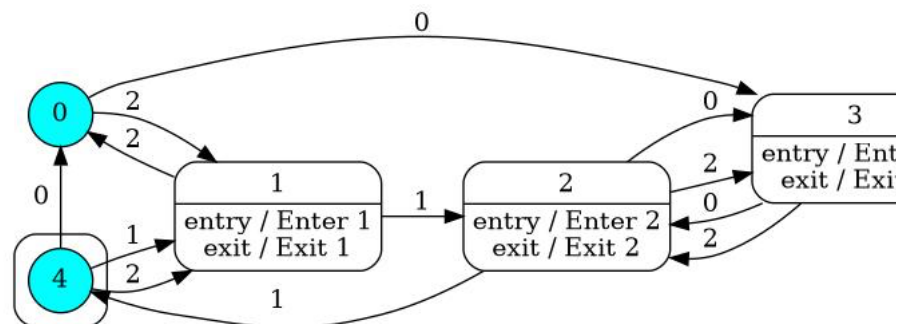
```
#include "main.h"

int f(int x, int u)
{ switch (x) {
  case 0: switch (u) {
    case 0: return (3);
    case 1: return (0);
    case 2: return (2);
    default: return (0); }
  case 1: switch (u) {
    case 0: return (1);
    case 1: return (1);
    case 2: return (2);
    default: return (0); }
  case 2: switch (u) {
    case 0: return (1);
    case 1: return (1);
    case 2: return (4);
    default: return (0); }
  case 3: switch (u) {
    case 0: return (4);
    case 1: return (3);
    case 2: return (0);
    default: return (0); }
  case 4: switch (u) {
    case 0: return (3);
    case 1: return (2);
    case 2: return (0);
    default: return (0); }
  default: return (0); }
} // f()
```

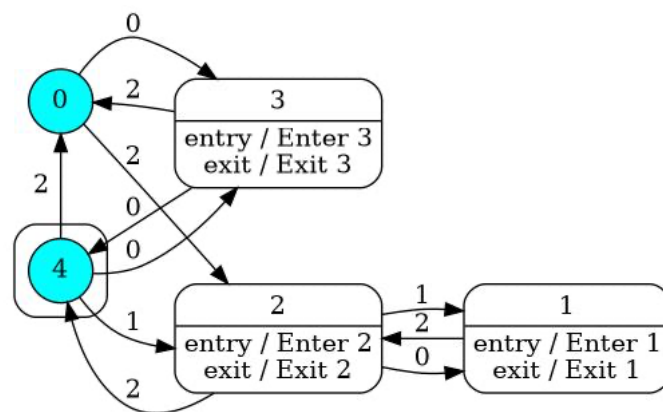
Quale dei seguenti UML state diagram rappresenta correttamente la macchina a stati finiti definita dalla funzione di transizione definita sopra?

Scegli un'alternativa:

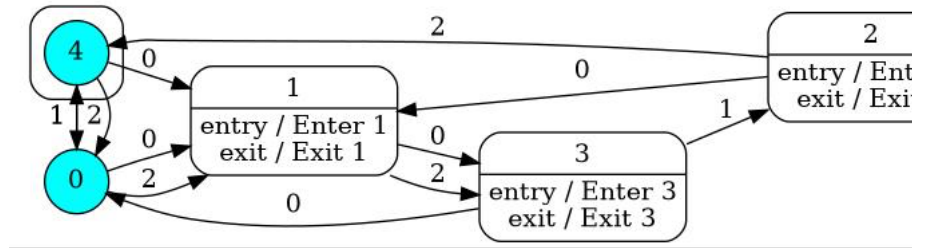
☒ 1.



☐ 2.

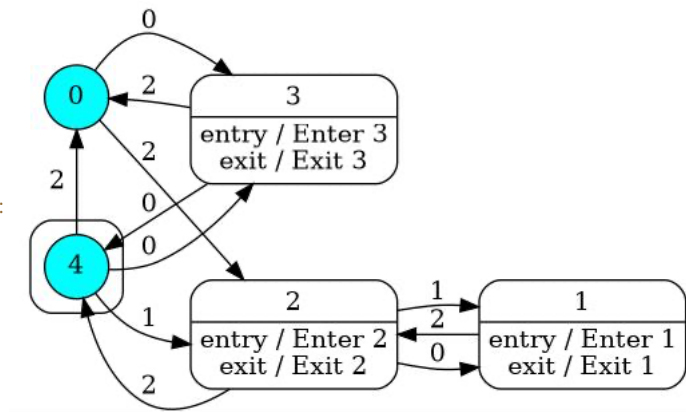


○ 3.



Your answer is incorrect.

La risposta corretta è:

**Domanda 48**

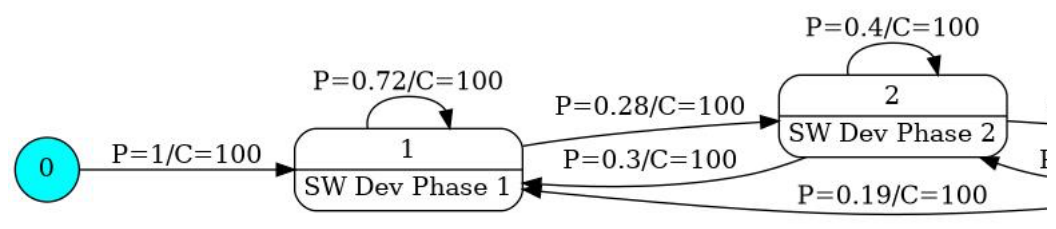
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un **Markov Decision Process**, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per il valore atteso del numero di transizioni necessario per lasciare la "SW Dev Phase 2"?

Scegli un'alternativa:

- 1. 8.53253
 ○ 2. 3.69941
 ● 3. 1.66667 ✓ Correct.

Your answer is correct.

La risposta corretta è: 1.66667

Domanda 49

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

La variabile x rappresenta lo stato presente del sistema, la variabile x_1 rappresenta lo stato precedente ad x e la variabile x_2 lo stato precedente ad x_1 .

Si consideri il seguente requisito:

Il valore assoluto della differenza tra il valore dello stato presente e di quello precedente è al più 2.

Quale delle seguenti assert definisce questo requisito (cioè ritorna errore quando il requisito non è verificato) ?

- ☒ a. `assert( abs(x - x1) <= 2)` ☺
- ☐ b. `assert( abs(x - x1) >= 2)`
- ☐ c. `assert( (x - x1) <= 2)`

Your answer is correct.

La risposta corretta è:
`assert( abs(x - x1) <= 2)`

Domanda 50

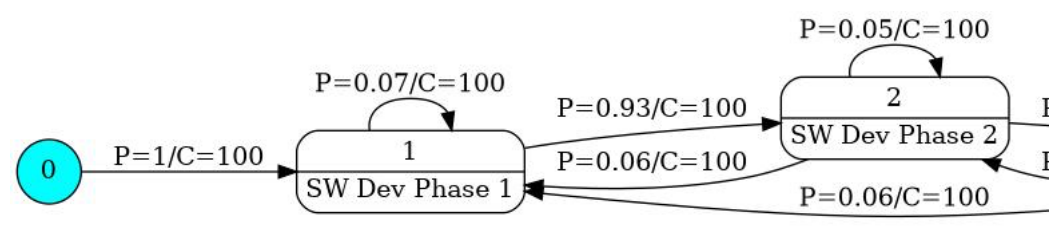
Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Un processo software può essere rappresentato con un *Markov Decision Process*, cioè, nel nostro caso, una catena di Markov i cui archi sono etichettati con la probabilità di transizione (P) ed il costo della transizione (C).

A seconda dell'aspetto che si vuole modellare il costo può essere usato per rappresentare sia un costo economico (ad esempio in EUR) che un costo temporale (ad esempio in mesi di lavorazione). Omettiamo quindi l'unità di misura sul costo.

Si consideri il processo software in figura.



Qual'è la migliore stima per la probabilità che il processo completi le sue attività con un costo di esattamente 500.000000 ?

Scegli un'alternativa:

- ☒ 1. 0.1169 ☺ Correct.
- ☐ 2. 0.0922591
- ☐ 3. 0.722351

Your answer is correct.

La risposta corretta è: 0.1169

